

PROGRAM - 1

Program Statement:

Implement a GUI based UDP Client program for Well Known Service – Echo.

Description of the program:

This is a UDP Client Program that uses the User Datagram Protocol (UDP) to send a message to a server at a specified IP address and port number (7). The client takes the input message from a text field, creates a DatagramSocket, and uses it to send the message as a DatagramPacket to the server. It then waits for a response from the server and receives it as a DatagramPacket, which is then displayed in a text area.

An Echo Service is a Service which always returns back the same message that it receives from the client.

Input:

IP Address –

10.2.0.5Port No. -- 7

Message -- XYZ

Output:

The message sent to the server is sent back as a response.

Packages Used:

- java.io.*;
- java.net.*;

IDE Used: Netbeans

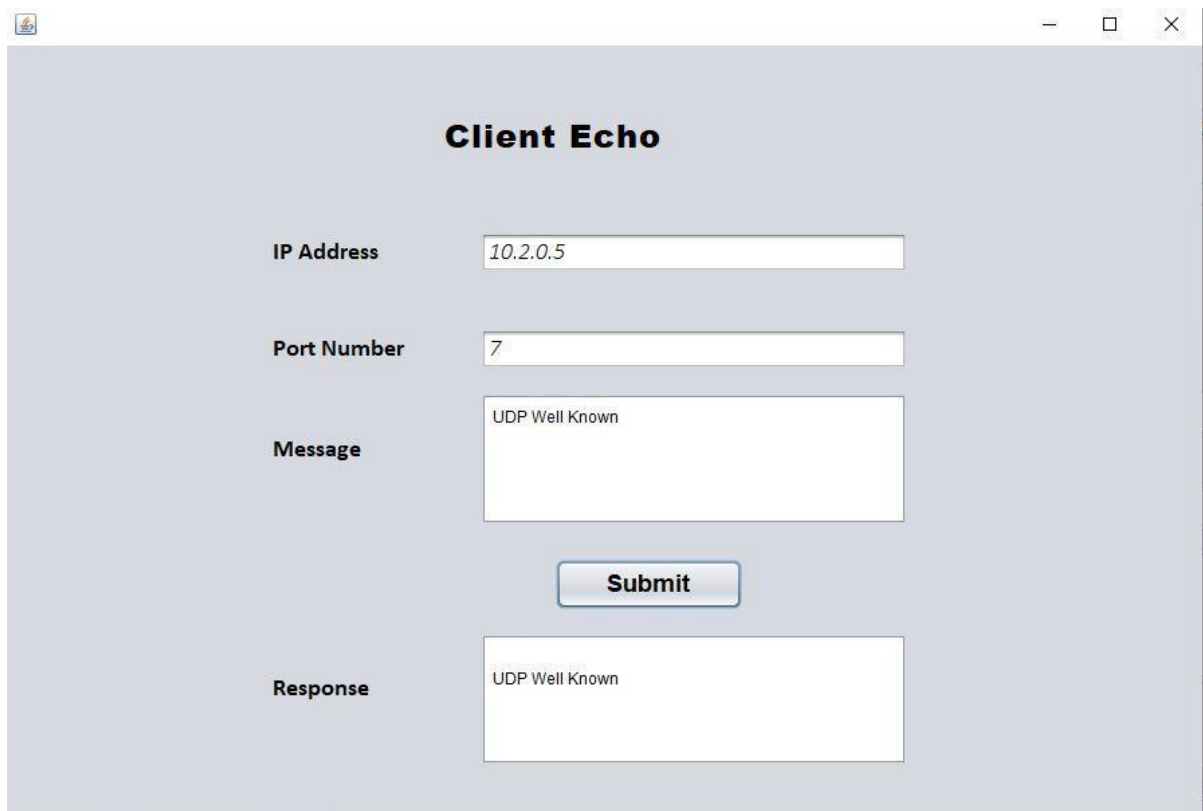
Classes and Methods Used:

- DatagramSocket
 - send()
 - receive()
 - close()
- DatagramPacket
 - DatagramPacket()
- InetAddress
 - getByName()

Program:

```
import java.net.*;
public class UDPClient extends javax.swing.JFrame {
public UDPClient() {
    initComponents(); // Assume your NetBeans GUI builder setup
}
private void SubmitButtonActionPerformed(java.awt.event.ActionEvent evt) {
    String ipAddress = IPField.getText().trim();
    int portNo = Integer.parseInt(PortField.getText().trim());
    String message = Mesg.getText().trim();
    try {
        // Create socket
        DatagramSocket ds = new DatagramSocket();
        // Send message
        byte[] sndmsg = message.getBytes();
        DatagramPacket sndPacket = new DatagramPacket(sndmsg, sndmsg.length,
            InetAddress.getByName(ipAddress), portNo);
        ds.send(sndPacket);
        // Receive response
        byte[] rcvmsg = new byte[1024];
        DatagramPacket rcvPacket = new DatagramPacket(rcvmsg, rcvmsg.length);
        ds.receive(rcvPacket);
        // Extract actual message
        String response = new String(rcvPacket.getData(), 0, rcvPacket.getLength());
        // Display on GUI
        Resp.append("Received: " + response + "\n");
        ds.close();
    } catch (Exception e) {
        Resp.append("Error: " + e.getMessage() + "\n");
        e.printStackTrace();
    }
}
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    /* Create and display the form */
    java.awt.EventQueue.invokeLater (new Runnable() {
        public void run() {
            new UDPClient().setVisible(true);
        }
    });
}
// GUI components declared here (IPField, portField, Mesg, Resp)
}
```

Output Images:



Client Echo

IP Address

Port Number

Message

Submit

Response

PROGRAM - 2

Program Statement:

Implement a GUI based TCP Client program for Well Known Service – Echo.

Description of the program:

This is a TCP Client Program that uses the Transmission Control Protocol (TCP) to send a message to a server at a specified IP address and port number (7). The client takes the input message from a text field, creates a Socket, and uses it to send the message on its DataOutputStream to the server. It then waits for a response from the server and receives it in DataInputStream, which is then displayed in a text area.

An Echo Service is a Service which always returns back the same message that it receives from the client.

Input:

IP Address –

10.2.0.5 Port No. --

7

Message -- XYZ

Output:

The message sent to the server is sent back as a response.

Packages Used:

- java.io.*;
- java.net.*;

IDE Used: Netbeans

Classes and Methods Used:

- Socket
 - Socket()
 - getInputStream()
 - getOutputStream()
 - close()
- DataInputStream
 - readUTF()
- DataOutputStream
 - writeUTF()
- InetAddress
 - getByName()

Program:

```
import java.io.*;
import java.net.*;
public class TCPClient extends javax.swing.JFrame {
public TCPClient() {
    initComponents(); // Assume your NetBeans GUI builder setup
}
private void SubmitButtonActionPerformed(java.awt.event.ActionEvent evt) {
    String ipAddress = IPField.getText().trim();
    int portNo = Integer.parseInt(PortField.getText().trim());
    String message = Mesg.getText().trim();
    try {
        // Send message
        Socket s = new Socket(InetAddress.getByName(ipAddress), portNo);

        DataOutputStream dos = new DataOutputStream(s.getOutputStream());
        dos.writeUTF(message);
        //Receive Message
        DataInputStream dis = new DataInputStream(s.getInputStream());
        String response = dis.readUTF();
        Resp.append("Echo: " + response + "\n");
        s.close();

    } catch (Exception e) {
        Resp.append("Error: " + e.getMessage() + "\n");
        e.printStackTrace();
    }
}
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new TCPClient().setVisible(true);
        }
    });
}
// GUI components declared here (IPField, portField, Mesg, Resp)
// Variables declaration - do not modify
private javax.swing.JTextField IPField;
private javax.swing.JTextField Mesg;
private javax.swing.JTextField PortField;
private javax.swing.JTextArea Resp;
private javax.swing.JButton SubmitButton;
private javax.swing.JLabel jLabel1;
private javax.swing.JLabel jLabel2;
private javax.swing.JLabel jLabel3;
private javax.swing.JLabel jLabel4;
private javax.swing.JScrollPane jScrollPane1;
// End of variables declaration
}
```

Output Images:



The image shows a screenshot of a web application window titled "Client Echo". The window has a light gray background and standard window controls (minimize, maximize, close) in the top right corner. The application contains the following elements:

- IP Address:** A text input field containing the value "10.2.0.5".
- Port Number:** A text input field containing the value "7".
- Message:** A text area containing the value "well known".
- Submit:** A blue button with the text "Submit".
- Response:** A text area containing the output "meeee" and "well known" on separate lines.

PROGRAM - 3

Program Statement:

Implement a GUI based UDP Client – Server program for Echo Service.

Description of the program:

This is a UDP Client-Server Echo Service Program that creates a Client Application that uses the User Datagram Protocol (UDP) to send a message to a server at a specified IP address and port number. The client takes the input message from a text field, the IP address and port number from text fields as well, creates a DatagramSocket, and uses it to send the message as a DatagramPacket to the server. It then waits for a response from the server and receives it as a DatagramPacket, which is then displayed in a text area.

An Echo Service is a Service which always returns back the same message that it receives from the client.

Input:

IP Address – localhost

Port No. – 50xx

Message -- XYZ

Output:

The message sent to the server is sent back as a response.

Packages Used:

- java.io.*;
- java.net.*;

IDE Used: Netbeans

Classes and Methods Used:

- DatagramSocket
 - send()
 - receive()
 - close()
- DatagramPacket
 - DatagramPacket()
- InetAddress
 - getByName()

Program:

Client:

```
package udpapplication;
import java.io.*;
import java.net.*;
import javax.swing.*; // GUI components (frame, text fields etc.)
public class UDPClient extends javax.swing.JFrame {
    public UDPClient() {
        initComponents(); // Assume your NetBeans GUI builder setup
    }
    @SuppressWarnings("unchecked")
    private void SendButtonActionPerformed(java.awt.event.ActionEvent evt) {
        // Read input from GUI fields
        String ipAddress = IPField.getText();
        int portNo = Integer.parseInt(portField.getText());
        String message = MessageField.getText();
        DatagramSocket ds = null;
        try {
            // Create an unbound DatagramSocket (OS chooses local port)
            ds = new DatagramSocket();
            // Convert message to bytes (platform default charset; consider UTF-8)
            byte[] sndmsg = message.getBytes();
            // Create packet addressed to server
            DatagramPacket sndPacket =
                new DatagramPacket(sndmsg, sndmsg.length, InetAddress.getByName(ipAddress),
                    portNo);
            // Send packet
            ds.send(sndPacket);
            // Prepare a buffer to receive response
            byte[] rcvmsg = new byte[1024]; // larger buffer
            DatagramPacket rcvPacket = new DatagramPacket(rcvmsg, rcvmsg.length);
            // Receive (this call blocks until a packet arrives or socket times out)
            ds.receive(rcvPacket);
            // Extract exact bytes received and convert to String (use length!)
            String response = new String(rcvPacket.getData(), 0, rcvPacket.getLength());
            ServerResponseArea.append("Received: " + response + "\n");
        } catch (Exception e) {
            e.printStackTrace();
            ServerResponseArea.append("Error: " + e.getMessage() + "\n");
        } finally {
            if (ds != null && !ds.isClosed()) ds.close();
        }
    }
}
```


Server:

```
package udpapplication;
import java.io.*;
import java.net.*;
import javax.swing.*; // GUI components
public class UDPServer extends javax.swing.JFrame implements Runnable {
    public Udp_Ser() {
        initComponents();
    }
    @SuppressWarnings("unchecked")
    private volatile boolean running = false;
    private void StartButtonActionPerformed(java.awt.event.ActionEvent evt) {
        StartButton.setEnabled(false);
        MessageArea.append("Server starting...\n");
        Thread thread = new Thread(this, "server");
        running = true;
        thread.start();
    }
    @Override
    public void run() {
        String ipAddress = IPField.getText();
        int portNo = Integer.parseInt(PortField.getText());
        DatagramSocket ds = null;
        try {
            // Bind socket to given local address and port
            ds = new DatagramSocket(portNo, InetAddress.getByName(ipAddress));
            MessageArea.append("Server started on " + ipAddress + ":" + portNo + "\n");
            byte[] rcvMessage = new byte[1024];
            while (running) {
                DatagramPacket rcvPacket = new DatagramPacket(rcvMessage, rcvMessage.length);
                // Blocks until a packet is received
                ds.receive(rcvPacket);
                InetAddress clientIP = rcvPacket.getAddress();
                int clientPort = rcvPacket.getPort();
                String received = new String(rcvPacket.getData(), 0, rcvPacket.getLength());
                MessageArea.append("Client " + clientIP + ":" + clientPort + " -> " + received + "\n");
                // Echo back the same bytes
                byte[] sndmsg = received.getBytes();
                DatagramPacket sndPacket = new DatagramPacket(sndmsg, sndmsg.length, clientIP, clientPort);
                ds.send(sndPacket);
            }
        } catch (Exception e) {
            e.printStackTrace();
            MessageArea.append("Server error: " + e.getMessage() + "\n");
        } finally {
            if (ds != null && !ds.isClosed()) ds.close();
            MessageArea.append("Server stopped.\n");
            StartButton.setEnabled(true);
        }
    }
    // A method to stop the server thread politely
    public void stopServer() {
        running = false;
    }
}
```

```

    }
}
public static void main(String args[]) {
    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new Udp_Ser().setVisible(true);

        }
    });
}

```

1. Run the server class first. Enter `localhost` (or your local IP) and port `5000` (for example) in the GUI, click Start.
2. Run the client class. Enter server address `localhost` and port `5000`. Type a message, click Send.
3. The server MessageArea should display the incoming message; the client `ServerResponseArea` should display the echoed message

Output Images:

The image displays two side-by-side windows from a network application. The left window is titled 'UDP Client' and contains three input fields: 'IP Address' with the value 'localhost', 'Port No' with the value '5041', and 'Message' with the value 'udpcs'. Below these fields is a 'Send Request' button. At the bottom, an 'Output' label is next to a text area containing the text 'udpcs'. The right window is titled 'UDP Server' and contains two input fields: 'IP Address' with the value 'localhost' and 'Port no' with the value '5041'. Below these is a 'Start Server' button. At the bottom, a 'Message' label is next to a text area containing the text 'Server listening...' and 'Client [127.0.0.1]: udpcs'.

UDP Client

IP Address: localhost

Port No: 5041

Message: udpcs

Send Request

Output: udpcs

UDP Server

IP Address: localhost

Port no: 5041

Start Server

Message: Server listening...
Client [127.0.0.1]: udpcs

PROGRAM - 4

Program Statement:

Implement a GUI based TCP Client – Server program for Echo Service.

Description of the program:

The is a Java Program that creates a Client Application that uses the Transmission Control Protocol (TCP) to send a message to a server at a specified IP address and port number. The client takes the input message from a text field, the IP address and port number from text fields as well, creates a Socket and connects it to the server using the IP address and port number. Then the client creates DataInputStream and DataOutputStream to read and write the data on the socket respectively. It sends the message to the server by writing it to the DataOutputStream, then reads the response from the server by reading it from the DataInputStream and displays it in a text area. The client then closes the socket.

An Echo Service is a Service which always returns back the same message that it receives from the client.

Input:

IP Address – localhost
Port No. – 50xx
Message -- XYZ

Output:

The message sent to the server is sent back as a response.

Packages Used:

- java.io.*;
- java.net.*;

IDE Used: Netbeans

Classes and Methods Used:

- Socket
 - Socket()
 - getInputStream()
 - getOutputStream()
 - close()
- DataInputStream
 - readUTF()
- DataOutputStream
 - writeUTF()
- ServerSocket
 - ServerSocket()
 - accept()
- InetAddress
 - getByName()

Program:

Client:

```
import java.net.*;
import java.io.*;
public class TCP_Client extends javax.swing.JFrame {
public TCP_Client() {
initComponents();
}
private void SubmitButtonActionPerformed(java.awt.event.ActionEvent evt) {
// TODO add your handling code here:
String ip = IP_Field.getText();
int port = Integer.parseInt(Port_Field.getText());
String msg = Mesg_Field.getText();
try{
Socket s = new Socket(InetAddress.getByName(ip), port);
DataInputStream dis = new DataInputStream(s.getInputStream());
DataOutputStream dos = new DataOutputStream(s.getOutputStream());
dos.writeUTF(msg);
String response = dis.readUTF();
Resp_Field.append(msg + "\n");
s.close();
}
catch(Exception e){
e.printStackTrace();
}
}
```

Server Code:

```
import java.net.*;
import java.io.*;
import java.lang.Runnable;
public class TCP_Server extends javax.swing.JFrame implements Runnable{
private void StartBtnActionPerformed(java.awt.event.ActionEvent evt) {
// TODO add your handling code here:
Thread t = new Thread(this, "Server");
t.start();
StartBtn.setEnabled(false);
Ser_Resp.append("Server Listening\n");
}
@Override
public void run() {
String ip = IP_Field.getText();
int port = Integer.parseInt(Port_Field.getText());
try{
ServerSocket ss = new ServerSocket(port, 5, InetAddress.getByName(ip));
while(true){
Socket s = ss.accept();
DataInputStream dis = new DataInputStream(s.getInputStream());
DataOutputStream dos = new DataOutputStream(s.getOutputStream());
String req = dis.readUTF();
Ser_Resp.append(req + "\n");
dos.writeUTF(req);
s.close();
}
}
catch(Exception e){ e.printStackTrace();
}
}
```

Output Images:

The image displays two side-by-side windows from a Java-based application. The left window is titled 'TCP Client' and contains three input fields: 'IP Address' with the value 'localhost', 'Port No' with the value '5041', and 'Message' with the value 'new'. Below these fields is a 'Send Request' button. At the bottom, an 'Output' text area displays the received messages 'amazing' and 'new'. The right window is titled 'TCP Server' and contains two input fields: 'IP Address' with the value 'localhost' and 'Port no' with the value '5041'. Below these is a 'Start Server' button. At the bottom, a 'Message' text area displays the server's log: 'Server listening....', 'Client [/127.0.0.1] : amazing', 'Client [/127.0.0.1] : amazing', and 'Client [/127.0.0.1] : new'. Both windows have standard Windows-style title bars with minimize, maximize, and close buttons.

TCP Client

IP Address

Port No

Message

Output
amazing
new

TCP Server

IP Address

Port no

Message
Server listening....
Client [/127.0.0.1] : amazing
Client [/127.0.0.1] : amazing
Client [/127.0.0.1] : new

PROGRAM - 5

Program Statement:

Implement a GUI based Client – Server program for Bulletin Board Application.

Description of the program:

The is a Java Program that creates a Client Application that connects to a Bulletin Board Server using the Transmission Control Protocol (TCP) at a specified IP address and port number. The client takes the input message from a text field, the IP address and port number from text fields as well, creates a Socket and connects it to the server using the IP address and port number. Then the client creates DataOutputStream to write the data on the socket. It sends the message to the server by writing it to the DataOutputStream, the server reads the message from the client and displays it in a text area. The client then closes the socket.

A Bulletin Board Application is an application dedicated to the sharing or exchange of messages on a network.

Input:

IP Address – localhost

Port No. – 50xx

Message -- XYZ

Output:

The messages sent by multiple clients to bulletin server are displayed at the server.

Packages Used:

- java.io.*;
- java.net.*;

IDE Used: Netbeans

Classes and Methods Used:

- Socket
 - Socket()
 - getInputStream()
 - getOutputStream()
 - close()
- DataInputStream
 - readUTF()
- DataOutputStream
 - writeUTF()
- ServerSocket
 - ServerSocket()
 - accept()
- InetAddress
 - getByName()

Program:

Client:

```
import java.net.*;import
java.io.*;
public class BulletinBoardClient extends javax.swing.JFrame {
public BulletinBoardClient() {
    initComponents();
}
private void SendButtonActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    String ip = IPField.getText();
    int port = Integer.parseInt(PortField.getText());
    String msg = MessageField.getText();
    try{
        Socket s = new Socket(InetAddress.getByName(ip), port);
        DataInputStream dis = new DataInputStream(s.getInputStream());
        DataOutputStream dos = new DataOutputStream(s.getOutputStream());
        dos.writeUTF(msg);
        String res = dis.readUTF();
        ResponseArea.append(res + "\n");
        s.close();
    }
    catch(Exception e){
        e.printStackTrace();
    }
}
```

Server:

```
import java.net.*;import
java.io.*;
import java.lang.Runnable;
public class BulletinBoardServer extends javax.swing.JFrame implements Runnable{
private void StartServerButtonActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    Thread t = new Thread(this, "Server"); t.start();
    StartServerButton.setEnabled(false);
    ResponseArea.append("Server Listening\n");
}
    @Override
    public void run(){
        String ip = IPField.getText();
        int port = Integer.parseInt(PortField.getText());
        try{
            ServerSocket ss = new ServerSocket(port, 5,InetAddress.getByName(ip));
            while(true){
                Socket s = ss.accept();
                DataInputStream dis = new DataInputStream(s.getInputStream());
                DataOutputStream dos = new DataOutputStream(s.getOutputStream());
                String announcement = dis.readUTF();
                ResponseArea.append("\n" + announcement);
                dos.writeUTF("Received Message");s.close();
            }
        }
        catch(Exception e){
            e.printStackTrace();
        }
}}}
```


Output Images:

The image shows two side-by-side Java Swing windows. The left window is titled "Client Bulletin" and contains input fields for "IP Address" (localhost), "Port Number" (5085), and "Message" (third message). Below the message field is a "SEND" button. The "Response" area at the bottom shows a scrollable text box with the message "The Message Has been displayed succe" repeated three times. The right window is titled "Server Bulletin" and contains input fields for "IP Address" (localhost) and "Port Number" (5085), with a "Start Server" button. The "RESPONSE" area at the bottom shows a scrollable text box with the text "Server is Listening" followed by three lines of log output: "[/127.0.0.1] first message", "[/127.0.0.1] second message", and "[/127.0.0.1] third message".

Client Bulletin

IP Address

Port Number

Message

Response

The Message Has been displayed succe
The Message Has been displayed succe
The Message Has been displayed succe

Server Bulletin

IP Address

Port Number

RESPONSE

Server is Listening
[/127.0.0.1] first message
[/127.0.0.1] second message
[/127.0.0.1] third message

PROGRAM - 6

Program Statement:

Implement a GUI based Client – Server program for Name Service/DNS Application.

Description of the program:

The is a Java Program that creates a Client Application that uses the Transmission Control Protocol (TCP) to send a URL to a DNS (Domain Name Service) Server at a specified IP address and port number. The client takes the input message from a text field, the IP address and port number from text fields as well, creates a Socket and connects it to the server using the IP address and port number. Then the client creates DataInputStream and DataOutputStream to read and write the data on the socket respectively. It sends the message to the server by writing it to the DataOutputStream, then reads the response from the server by reading it from the DataInputStream and displays it in a text area. The client then closes the socket.

A DNS server is a computer server that contains a database of public IP addresses and their associated hostnames, and in most cases serves to resolve, or translate, those names to IP addresses as requested.

Input:

IP Address – localhost
Port No. – 50xx
URL – www.abc.com

Output:

The Client sends a URL to the server and gets its Ip address as response.

Packages Used:

- java.io.*
- java.net.*

Classes and Methods Used:

- Socket
 - Socket()
 - getInputStream()
 - getOutputStream()
 - close()
- DataInputStream
 - readUTF()
- DataOutputStream
 - writeUTF()
- ServerSocket
 - ServerSocket()
 - accept()
- InetAddress
 - getByName()

Program:

Server:

```
import java.io.*;
import java.net.*;
import java.util.Scanner;
import java.lang.Runnable;
import java.util.HashMap;
import java.util.Map;
public class DNS_Ser extends javax.swing.JFrame implements Runnable {
    private Map<String, String> dnsTable = new HashMap<>();
    /**
     * Creates new form DNS_Ser
     */
    public DNS_Ser() {
        initComponents();
    }
    private void Start_BtnActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        Thread t = new Thread(this, "server");
        Start_Btn.setEnabled(false);
        Resp.append("Starting DNS server...\n");
        // Load DNS data once
        loadDNSData();
        t.start();
    }
    private void loadDNSData() {
        try {
            File f = new File("C:\\Users\\admin\\Desktop\\DS\\DNS_DATA.txt");
            Scanner scanner = new Scanner(f);
            while (scanner.hasNextLine()) {
                String[] tokens = scanner.nextLine().split(" ");
                if (tokens.length == 2) {
                    dnsTable.put(tokens[1].trim(), tokens[0].trim());
                }
            }
            scanner.close();
            Resp.append("DNS data loaded. Entries: " + dnsTable.size() + "\n");
        } catch (Exception e) {
            Resp.append("Error loading DNS data: " + e.getMessage() + "\n");
        }
    }
    @Override
    public void run() {
        String ipAddress = IP_Field.getText();
        int portNo = Integer.parseInt(Port_Field.getText());
        try (ServerSocket ss = new ServerSocket(portNo, 5, InetAddress.getByName(ipAddress))) {
            Resp.append("Server running on " + ipAddress + ":" + portNo + "\n");
            while (true) {
                Socket s = ss.accept();
                new Thread(() -> handleClient(s)).start();
            }
        } catch (Exception e) {
            Resp.append("Server error: " + e.getMessage() + "\n");
        }
    }
    private void handleClient(Socket s) {
        try (DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new DataOutputStream(s.getOutputStream())) {
```

```

        String url = dis.readUTF();
        Resp.append("Received URL: " + url + "\n");
        String result = dnsTable.getDefault(url, "Not Found");
        dos.writeUTF(result);
        Resp.append("Response sent: " + result + "\n\n");
    } catch (IOException e) {
        Resp.append("Client connection error: " + e.getMessage() + "\n");
    }
}
/**
 * @param args the command line arguments
 */
public static void main(String args[]) {
    /* Set the Nimbus look and feel */
    /* Create and display the form */
    java.awt.EventQueue.invokeLater(new Runnable() {
        public void run() {
            new DNS_Ser().setVisible(true);
        }
    });
}
}

```

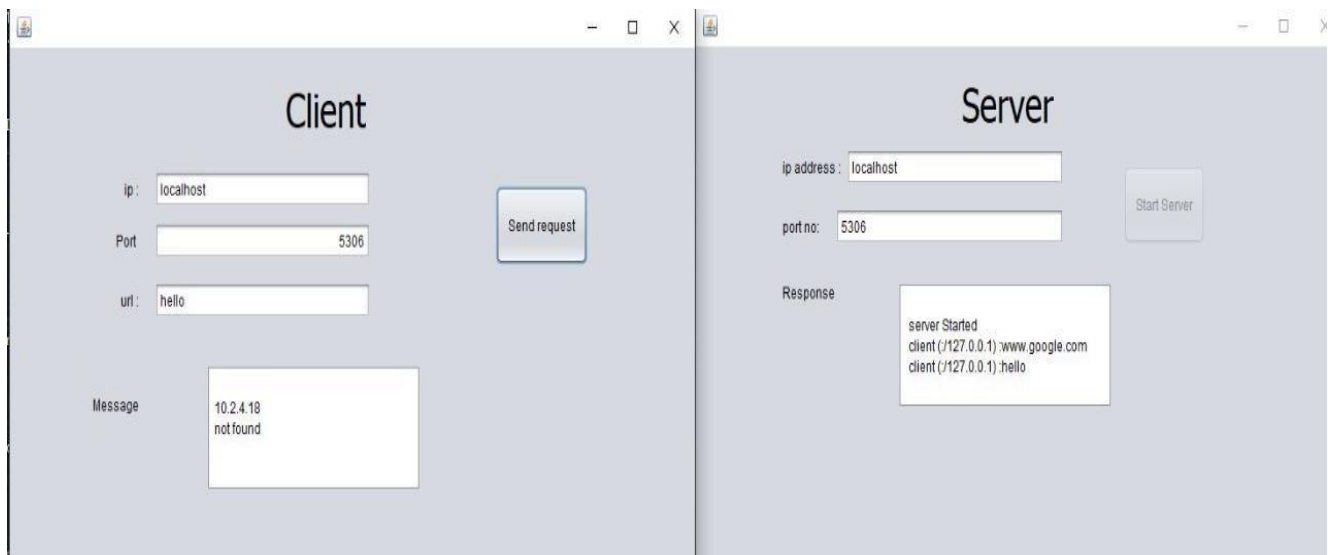
Client:

```

import java.net.*;
import java.io.*;
public class DNS_Cl extends javax.swing.JFrame {
    /**
     * Creates new form DNS_Cl
     */
    public DNS_Cl() {
        initComponents();
    }
    private void Send_BtnActionPerformed(java.awt.event.ActionEvent evt) {
        // TODO add your handling code here:
        String ipAddress = IP_Field.getText();
        int portNo = Integer.parseInt(Port_Field.getText());
        String url = URL_Field.getText();
        try{
            Socket s = new Socket(InetAddress.getByName(ipAddress), portNo);
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());
            DataInputStream dis = new DataInputStream(s.getInputStream());
            dos.writeUTF(url);
            Resp.append(url);
            String response = dis.readUTF();
            Resp.append(": " + response + "\n");
            s.close();
        }
        catch(Exception e)
        { e.printStackTrace();
        }
    }
}

```

Output Images:



text - Notepad

File Edit Format View Help

```
www.google.com 10.2.4.18  
www.mysql.com 10.2.4.19  
www.youtube.com 10.2.5.6
```

PROGRAM - 7

Program Statement:

Implement a RPC program for Echo Service.

Description of the program:

RPC (Remote Procedure Call) allows programs to call procedures located on other machines. When a process on machine A calls a procedure on machine B, the calling process on A is suspended, and execution of the called procedure takes place on B. Information can be transported from the caller to the callee in the parameters and can come back in the procedure result. No message passing at all is visible to the programmer. This method is known as Remote Procedure Call, or often just RPC. The steps of RPC are,

1. The client procedure calls the client stub in the normal way.
2. The client stub builds a message and calls the local operating system.
3. The client's as sends the message to the remote as.
4. The remote as gives the message to the server stub.
5. The server stub unpacks the parameters and calls the server.
6. The server does the work and returns the result to the stub.
7. The server stub packs it in a message and calls its local as.
8. The server's as sends the message to the client's as.
9. The client's as gives the message to the client stub.
10. The stub unpacks the result and returns to the client

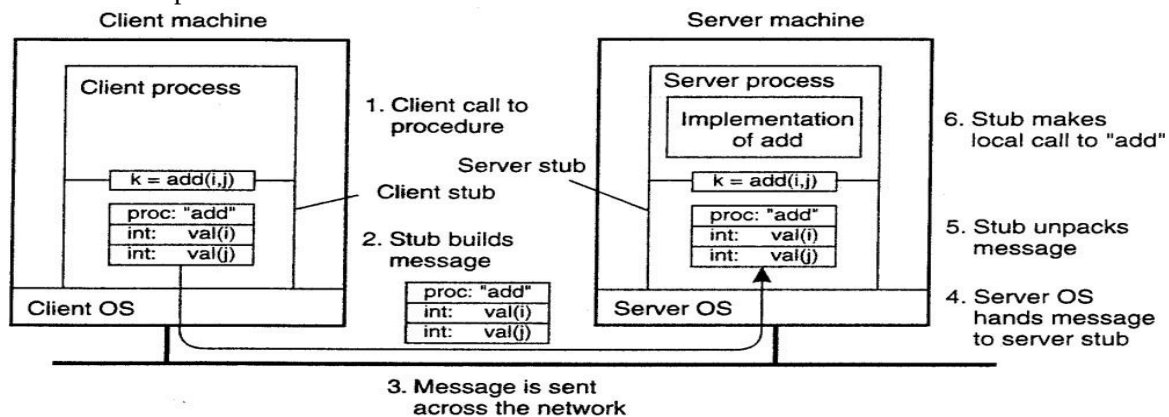


Figure 4-7. The steps involved in a doing a remote computation through RPC.

This is a program in c which uses Sun-RPC (Remote Procedure Call) library to implement Echo Service. Here the client sends a message by making a function call to a remote procedure present on the server. As an Echo Service, the server sends back the same message as a response.

Input:

A message to send to the server.

Output:

The message sent to server is sent back as response.

Program:

Specification file

//program name: echo.x

program ECHOPRG{

version ECHOER{

string echo(string)=1;

}=1;

}=0x20000001;

// compiling echo.x using rpcgen

\$ rpcgen -a echo.x

\$ ls

// seeing the list of files which gets created(6 files)

echo.h echo.x echo_client.c echo_clnt.c echo_server.c echo_svc.c Makefile.echo

// open echo_server.c to do changes.

//echo_server.c

/*

*** This is sample code generated by rpcgen.**

*** These are only templates and you can use them**

*** as a guideline for developing your own functions.**

***/**

#include "echo.h"

char **echo_1_svc(char **argp, struct svc_req *rqstp)

{

static char * result;

/*

*** insert server code here**

***/**

result=(char*)malloc(sizeof(char)*25);

strcpy(result,*argp);

return &result;

}

// open echo_client.c to make changes

//echo_client.c

/*

*** This is sample code generated by rpcgen.**

*** These are only templates and you can use them**

*** as a guideline for developing your own functions.**

***/**

#include "echo.h"

void echoprgr_1(char *host)//argument for ECHOPRG_1

{

CLIENT *clnt;

char * *result_1;

char * echo_1_arg;

#ifndef DEBUG

clnt = clnt_create (host, ECHOPRG, ECHOER, "udp");//used for creating client.can use udp or tcp

if (clnt == NULL) {

clnt_pcreateerror (host);

exit (1);

}

#endif /* DEBUG */

echo_1_arg=(char*)malloc(sizeof(char)*25);

printf("\nEnter a message:");

scanf("%s",echo_1_arg);

```

        result_1 = echo_1(&echo_1_arg, clnt); //for calling client stub
        if (result_1 == (char **) NULL) {
            clnt_perror (clnt, "call failed");
        }
        printf("%s", *(result_1));
#ifdef DEBUG
        clnt_destroy (clnt);
#endif /* DEBUG */
    }
    int main (int argc, char *argv[])
    {
        char *host;
        if (argc < 2) {
            printf ("usage: %s server_host\n", argv[0]);
            exit (1);
        }
        host = argv[1];
        echoprg_1 (host);
    exit (0);
    }

```

OUTPUT:

```

//compiling server and server stub program
$ cc echo_server.c echo_svc.c -o recho
$ ./recho&
[1] 9372
// compiling client and client stub program
$ cc echo_client.c echo_clnt.c
$ ./a.out 10.2.0.9
Enter the message: hello_welcome_to_lab
hello_welcome_to_lab
hello_welcome_to_lab

```


PROGRAM - 8

Program Statement:

Implement a RPC program for Reversing a given String.

Description of the program:

This is a program in c which uses Sun-RPC (Remote Procedure Call) library to implement Reverse of a Given String. Here the client sends a message by making a function call to a Remote Procedure present on the server. The server sends back the Reverse of the String as a Response.

Input:

A message to send to the server.

Output:

The message sent to server is reversed and sent back as response.

Program:

Specification file:

//program name: rev.x

```
program rev_prog{
version rev_ver{
    string revstr(string)=1;
    }=1;
}=0x20000022;
```

// compiling echo.x using rpcgen

\$ rpcgen -a rev.x

\$ ls

// seeing the list of files which gets created(6 files)

rev.h rev.x rev_client.c rev_clnt.c rev_server.c rev_svc.c Makefile.rev

// open rev_server.c to do changes.

//rev_server.c

/*

*** This is sample code generated by rpcgen.**
*** These are only templates and you can use them**
*** as a guideline for developing your own functions.**
***/**

#include "rev.h"

#include<string.h>

#include<stdlib.h>

char **revstr_1_svc(char **argp, struct svc_req *rqstp)

{

static char * result;

int i,j;

result=(char*)malloc(sizeof(char)*20);

for(j=0,i=strlen(*argp)-1;i>=0;i--,j++)

*(result+j)=(*argp+i);

*(result+j)='\0';

return &result;

}

// open rev_client.c to make changes

//rev_client.c

/*

*** This is sample code generated by rpcgen.**

```

* These are only templates and you can use them
* as a guideline for developing your own functions.
*/

#include "rev.h"
void rev_prog_1(char *host)
{
    CLIENT *clnt;
    char **result_1;
    char * revstr_1_arg;
#ifdef DEBUG
    clnt = clnt_create (host, rev_prog, rev_ver, "udp");
    if (clnt == NULL) {
        clnt_pcreateerror (host);exit (1);
    }
#endif /* DEBUG */
    revstr_1_arg=(char*)malloc(sizeof(char)*20);
    printf("\nEnter a string:");
    scanf("%s",revstr_1_arg);
    result_1 = revstr_1(&revstr_1_arg, clnt);
    if (result_1 == (char **) NULL) {
        clnt_perror (clnt, "call failed");
    }
    printf("result=%s",*result_1);
#ifdef DEBUG
    clnt_destroy (clnt);
#endif /* DEBUG */
}

int main (int argc, char *argv[])
{
    char *host;
    if (argc < 2) {
        printf ("usage: %s server_host\n", argv[0]);
        exit (1);
    }
    host = argv[1];
    rev_prog_1 (host);
    exit (0);
}

```

OUTPUT:

//compiling server and server stub program

\$ cc rev_server.c rev_svc.c -o rev

\$./rev&

[1] 9375

// compiling client and client stub program

\$ cc rev_client.c rev_clnt.c

\$./a.out 10.2.0.9

Enter the message: hello

olleh

PROGRAM - 9

Program Statement:

Implement a RMI program for Echo Service

Description of the program:

This program is a simple Remote Method Invocation (RMI) Application that allows a client to send a message to a server and receive the same message as a response identical to an Echo Service. The application consists of three parts: an interface, an implementation class, and a server and client class.

The EchoInterface interface defines a single Remote Method, echo(String s), which takes in a string as a parameter and returns the same string.

The EchoImpl class is the Implementation of the EchoInterface. The class extends UnicastRemoteObject, which is necessary for objects that are exported to use RMI. The echo method in this class takes in a string as a parameter, prints it out, and returns the same string.

The EchoServer class is the Server-Side Component of the Application. The main method of this class creates an instance of EchoImpl, creates an RMI registry on a specified port, and binds the EchoImpl object to a location in the registry.

The EchoClient class is the Client-Side Component of the Application. The main method of this class looks up the EchoImpl object in the RMI registry using the specified location, prompts the user for input and sends it to the echo method of the EchoImpl object, and prints out the response received from the server.

To run the application, the server must be started first, followed by the client. The server and client both take in three command-line arguments: the IP address of the server, the port number of the RMI registry, and the name to bind the EchoImpl object to in the registry.

Input:

A message to send to the server.

Output:

The message sent to server is sent back as response.

Packages Used:

- java.rmi.*
- java.io.*
- java.net.*
- java.rmi.server.*

Classes and Methods Used:

- LocateRegistry
 - createRegistry()
- Naming
 - bind()
 - lookup()
- Remote
- RemoteException
- MalformedURLException
- UnicastRemoteObject

Program:**Interface: Program Name: EchoInterface.java**

```
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface EchoInterface extends Remote { String echo(String s) throws RemoteException;
}
```

Implementation: Program Name: EchoImpl.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
public class EchoImpl extends UnicastRemoteObject implements EchoInterface{
    public EchoImpl() throws RemoteException{
    }
    public String echo(String s1)
    {
        System.out.println("REMOTE SERVICE: Remote Client Request Messageis : "+s1);
        return s1;
    }
}
```

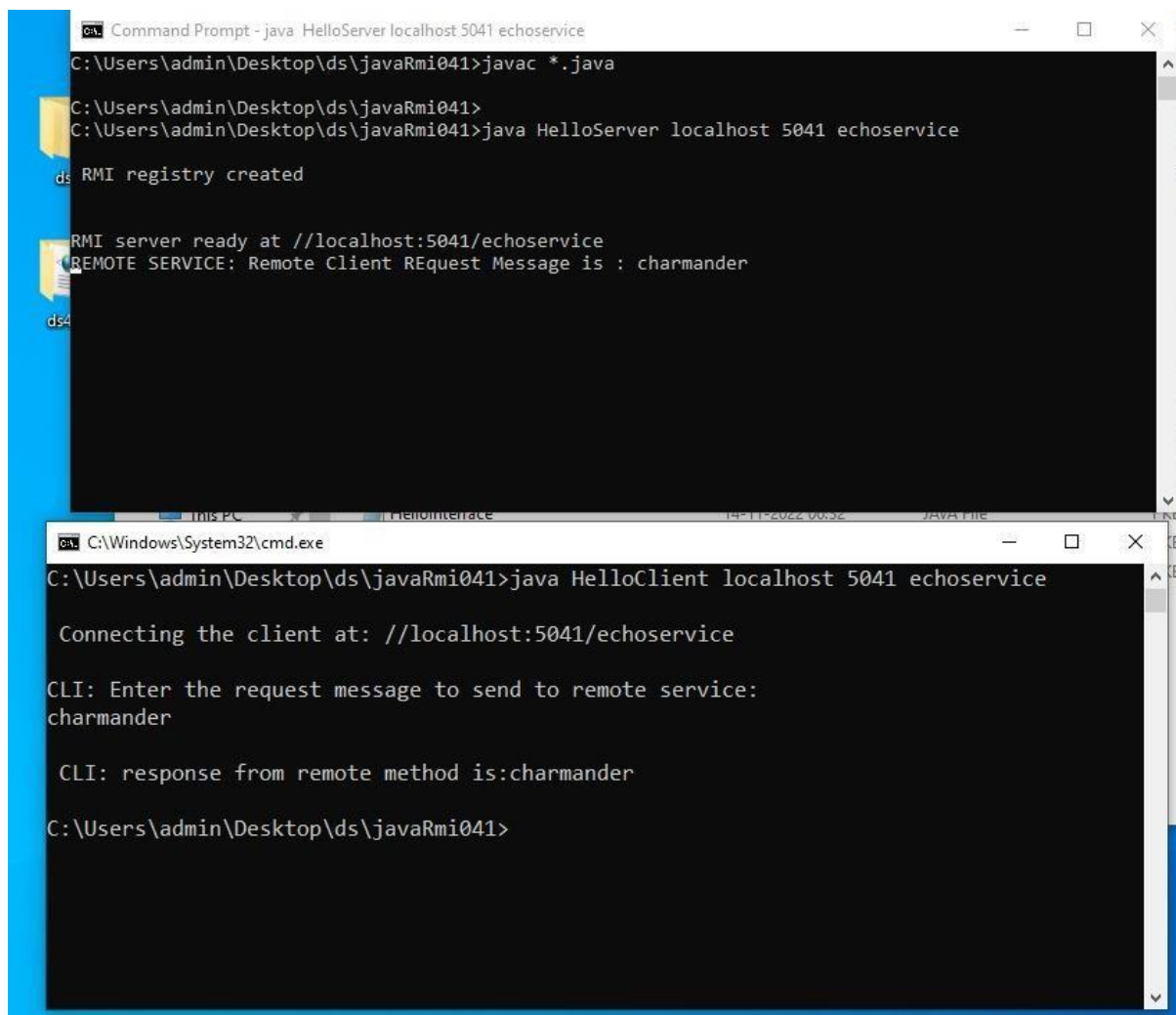
Server: Program Name: EchoServer.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;
import java.rmi.Naming;
import java.net.MalformedURLException;
import java.rmi.registry.LocateRegistry;
public class EchoServer{
    public EchoServer() throws RemoteException{
    }
    public static void main(String args[]) throws RemoteException
    {
        EchoImpl echoObj=new EchoImpl();
        int port=Integer.parseInt(args[1]);
        try{
            LocateRegistry.createRegistry(port);
            System.out.println("\n RMI registry created \n");
            String host=args[0];
            String bindLocation="//"+host+": "+port+"/"+args[2];
            Naming.bind(bindLocation,echoObj);
            System.out.println("\nRMI server ready at "+bindLocation);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

Client:

```
import java.rmi.*;
import java.io.*;
import java.net.MalformedURLException;
public class EchoClient {
    public static void main(String args[])
    {
        String connectLocation="//"+args[0]+":"+Integer.parseInt(args[1])+"/"+args[2];
        EchoInterface eintf=null;
        try{
            System.out.println("\n Connecting the client at:"+connectLocation);
            eintf=(EchoInterface)Naming.lookup(connectLocation);
            BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
            System.out.println("\nCLI: Enter the request message to sendto remote service:");
            String s=br.readLine();
            String response=eintf.echo(s);
            System.out.println("\n CLI: response from remote methodis:"+response);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

Output Images:



```
Command Prompt - java HelloServer localhost 5041 echoservice
C:\Users\admin\Desktop\ds\javaRmi041>javac *.java
C:\Users\admin\Desktop\ds\javaRmi041>
C:\Users\admin\Desktop\ds\javaRmi041>java HelloServer localhost 5041 echoservice
ds RMI registry created
RMI server ready at //localhost:5041/echoservice
REMOTE SERVICE: Remote Client REquest Message is : charmander
ds4

C:\Windows\System32\cmd.exe
C:\Users\admin\Desktop\ds\javaRmi041>java HelloClient localhost 5041 echoservice
Connecting the client at: //localhost:5041/echoservice
CLI: Enter the request message to send to remote service:
charmander
CLI: response from remote method is:charmander
C:\Users\admin\Desktop\ds\javaRmi041>
```

PROGRAM - 10

Program Statement:

Implement a RMI program for Reversing a given String.

Description of the program:

The program consists of an interface, an implementation class, and a server and client class.

The ReverseInterface interface defines a Single Remote method, reverseStr(String s), which takes in a string as a parameter and returns the Reverse of the String.

The ReverseImpl class is the implementation of the ReverseInterface. The class extends UnicastRemoteObject, which is necessary for objects that are exported to use RMI. The reverseStr method in this class takes in a string as a parameter, creates a StringBuilder object with the input string, reverses the string using the StringBuilder's reverse() method, and returns the reversed string.

The ReverseServer class is the Server-Side Component of the Application. The main method of this class creates an instance of ReverseImpl, creates an RMI registry on the specified port, binds the object to the registry, and makes it available for remote clients to access.

The ReverseClient class is the Client-Side Component of the Application. The main method of this class looks up the remote object on the specified IP address, port and service name, prompts the user to enter a string, calls the reverseStr method on the remote object, and prints the response.

The client and server communicate using RMI, which allows the client to invoke methods on the remote object as if they were local methods. The client sends the string to the server, the server reverses the string and sends the reversed string back to the client, which then prints it.

Input:

A message to send to the server.

Output:

The message sent to server is reversed and sent back as response.

Packages Used:

- java.rmi.*
- java.io.*
- java.net.*
- java.rmi.server.*

Classes and Methods Used:

- LocateRegistry
 - createRegistry()
- Naming
 - bind()
 - lookup()
- Remote
- RemoteException
- MalformedURLException
- UnicastRemoteObject

Program:**Interface: Filename:ReverseInterface.java**

```
import java.rmi.Remote;
import java.rmi.RemoteException;
public interface ReverseInterface extends Remote{ String reverseStr(String s) throws RemoteException;
}
```

Implementation: Filename:ReverseImpl.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
public class ReverseImpl extends UnicastRemoteObject implements ReverseInterface{
    public ReverseImpl() throws RemoteException{
    }
    public String reverseStr(String s1)
    {
        System.out.println("REMOTE SERVICE: Remote Client Request Messageis : "+s1);
        StringBuilder sb=new StringBuilder(s1); String
        response=sb.reverse().toString();
        return response;
    }
}
```

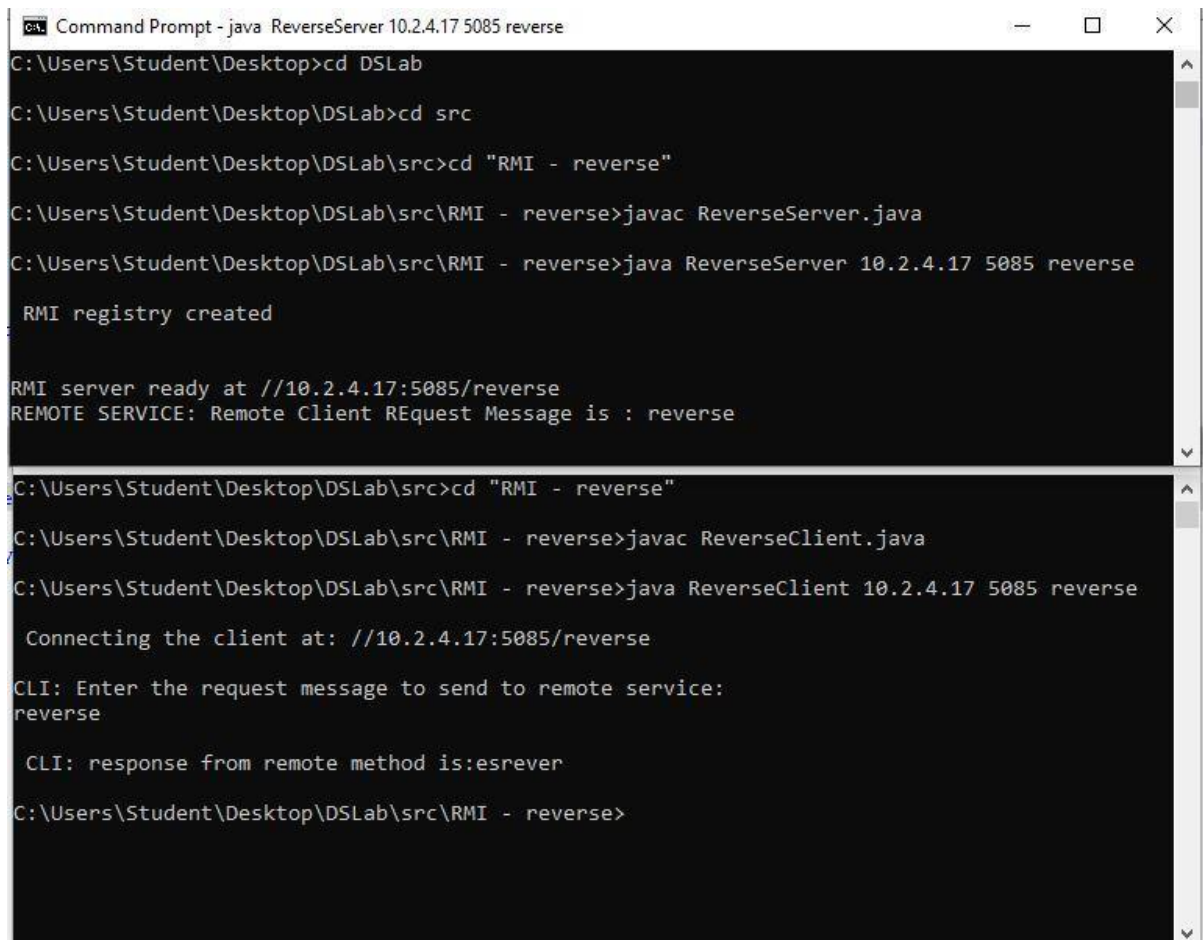
Server: Filename:ReverseServer.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;
import java.rmi.Naming;
import java.net.MalformedURLException;
import java.rmi.registry.LocateRegistry;
public class ReverseServer{
    public ReverseServer() throws RemoteException{
    }
    public static void main(String args[]) throws RemoteException
    {
        ReverseImpl reverseObj=new ReverseImpl();
        int port=Integer.parseInt(args[1]);
        try{
            LocateRegistry.createRegistry(port);
            System.out.println("\n RMI registry created \n");
            String host=args[0];
            String bindLocation="//"+host+": "+port+"/"+args[2];
            Naming.bind(bindLocation,reverseObj);
            System.out.println("\nRMI server ready at "+bindLocation);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}
```


Client: Filename:ReverseClient.java

```
import java.rmi.*;
import java.io.*;
import java.net.MalformedURLException;
public class ReverseClient {
    public static void main(String args[])
    {
        String connectLocation="//"+args[0]+":"+Integer.parseInt(args[1])+"/"+args[2];
        ReverseInterface revintf=null;
        try{
            System.out.println("\n Connecting the client at:"+connectLocation);
            revintf=(ReverseInterface)Naming.lookup(connectLocation);
            BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
            System.out.println("\nCLI: Enter the request message to sendto remote service:");
            String s=br.readLine();
            String response=revintf.reverseStr(s);
            System.out.println("\n CLI: response from remote methodis:"+response);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

Output Images:



The image displays two screenshots of a Windows Command Prompt window. The title bar of the window reads "Command Prompt - java ReverseServer 10.2.4.17 5085 reverse".

The first screenshot shows the following commands and output:

```
C:\Users\Student\Desktop>cd DSLab
C:\Users\Student\Desktop\DSLAb>cd src
C:\Users\Student\Desktop\DSLAb\src>cd "RMI - reverse"
C:\Users\Student\Desktop\DSLAb\src\RMI - reverse>javac ReverseServer.java
C:\Users\Student\Desktop\DSLAb\src\RMI - reverse>java ReverseServer 10.2.4.17 5085 reverse

RMI registry created

RMI server ready at //10.2.4.17:5085/reverse
REMOTE SERVICE: Remote Client RRequest Message is : reverse
```

The second screenshot shows the following commands and output:

```
C:\Users\Student\Desktop\DSLAb\src>cd "RMI - reverse"
C:\Users\Student\Desktop\DSLAb\src\RMI - reverse>javac ReverseClient.java
C:\Users\Student\Desktop\DSLAb\src\RMI - reverse>java ReverseClient 10.2.4.17 5085 reverse

Connecting the client at: //10.2.4.17:5085/reverse
CLI: Enter the request message to send to remote service:
reverse

CLI: response from remote method is:esrever
C:\Users\Student\Desktop\DSLAb\src\RMI - reverse>
```

PROGRAM - 11

Program Statement:

Implement a GUI based FTP Client program to list directory contents/ list files.

Description of the program:

FTP (File Transfer Protocol) is a network protocol for transmitting files between computers over Transmission Control Protocol/Internet Protocol (TCP/IP) connections. Within the TCP/IP suite, FTP is considered an application layer protocol.

FTP File Listing Operation connects to an FTP Server and generates a list of files present in the current working directory of the FTP Login.

This code is for a Java Swing Application that connects to an FTP server using the Apache Commons Net library. When the user clicks the "List Files" button, the program retrieves the values entered in the IP, username, and password fields. It then creates an instance of the FTPClient class from the Apache Commons library, connects to the server, and attempts to log in with the provided credentials. If the login is successful, it retrieves a list of files in the current directory and displays them in a dropdown box. It also appends the file names and other status messages to a text area called "statusWindowArea". Finally, it logs out and disconnects from the server. Any exceptions that may occur during the process are caught and printed to the console.

Input:

IP Address – 10.2.0.5

Username – cse22xx

Password – xxxxxxxx

Output:

The files in the ftp server login are listed in the output window text area.

Packages Used:

- java.io.*
- org.apache.commons.net.ftp.*

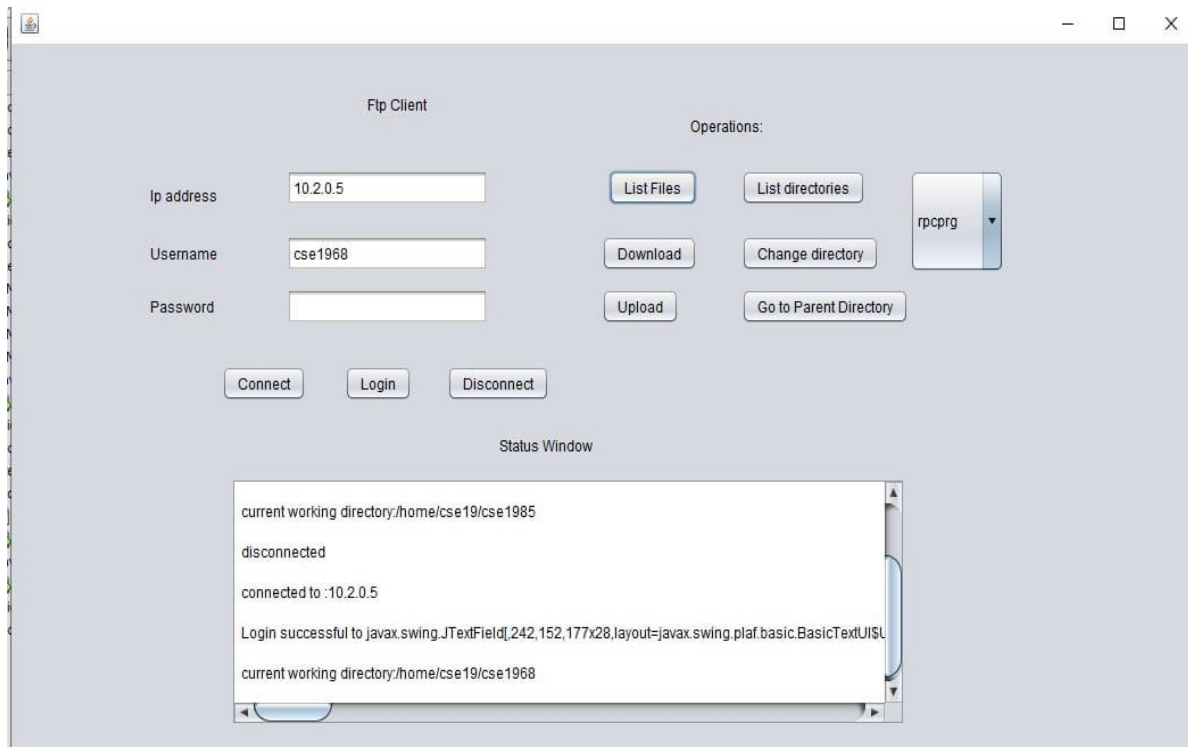
Classes and Methods Used:

- FTPClient
 - FTPClient()
 - connect()
 - getReplyCode()
 - login()
 - listFiles()
 - logout()
 - disconnect()
- FTPReply
 - isPositiveCompletion()
- FTPFile
 - getName()

Program:

```
package ftpserverclientby;
import java.io.*;
import org.apache.commons.net.ftp.*;
import org.apache.commons.net.ftp.FTPClient;
public class FTPClientInterface extends javax.swing.JFrame {
    private void listFilesButtonActionPerformed(java.awt.event.ActionEvent evt) {
        String ip = IPField.getText();
        String username = usernameField.getText();
        String password = new String(passwordField.getPassword());
        FTPClient ftpc = new FTPClient();
        try {
            ftpc.connect(ip);
            int status = ftpc.getReplyCode();
            if (FTPReply.isPositiveCompletion(status)) {
                statusWindowArea.append("Connected to " + ip + "\n");
                if (ftpc.login(username, password)) {
                    statusWindowArea.append("User " + username + " logged in to " + ip + "\n");
                    statusWindowArea.append("Current directory is " + ftpc.printWorkingDirectory() + "\n");
                    FTPFile[] files = ftpc.listFiles();
                    dropdownBox.removeAllItems();
                    for (FTPFile file : files) {
                        dropdownBox.addItem(file.getName());
                        statusWindowArea.append("> " + file.getName() + "\n");
                    }
                    ftpc.logout();
                    statusWindowArea.append("User " + username + " logged out.\n");
                }
            }
            else {
                statusWindowArea.append("Login failed: Invalid credentials\n");
            }
            ftpc.disconnect();
        }
        else {
            statusWindowArea.append("Connection failed to " + ip + "\n");
            ftpc.disconnect();
        }
    }
    catch (Exception e) {
        statusWindowArea.append("Error: " + e.getMessage() + "\n");
        try {
            if (ftpc.isConnected()) {
                ftpc.disconnect();
            }
        }
        catch (IOException ex) { }
    }
}
```

Output Images:



PROGRAM - 12

Program Statement:

Implement a GUI based FTP Client program to download a file.

Description of the program:

FTP File Download Operation connects to an FTP Server and downloads the specified file from the specified path on the remote system to the local system.

The code you provided is for a Java Swing based FTP client. The main functionalities are connecting to an FTP server, logging in, listing files, and downloading a file from the server. The “listFilesButtonActionPerformed” method is triggered when the "List Files" button is pressed, it connects to the FTP server using the IP, username and password provided by the user and lists all the files in the current directory. The “downloadButtonActionPerformed” method is triggered when the "Download" button is pressed, it allows the user to select a file from the list of files in the current directory and download it to the local machine.

Input:

IP Address – 10.2.0.5

Username – cse22xx

Password – xxxxxxxx

Output:

The files in the ftp server login are listed and download operation is performed for a file.

Packages Used:

- java.io.*
- org.apache.commons.net.ftp.*

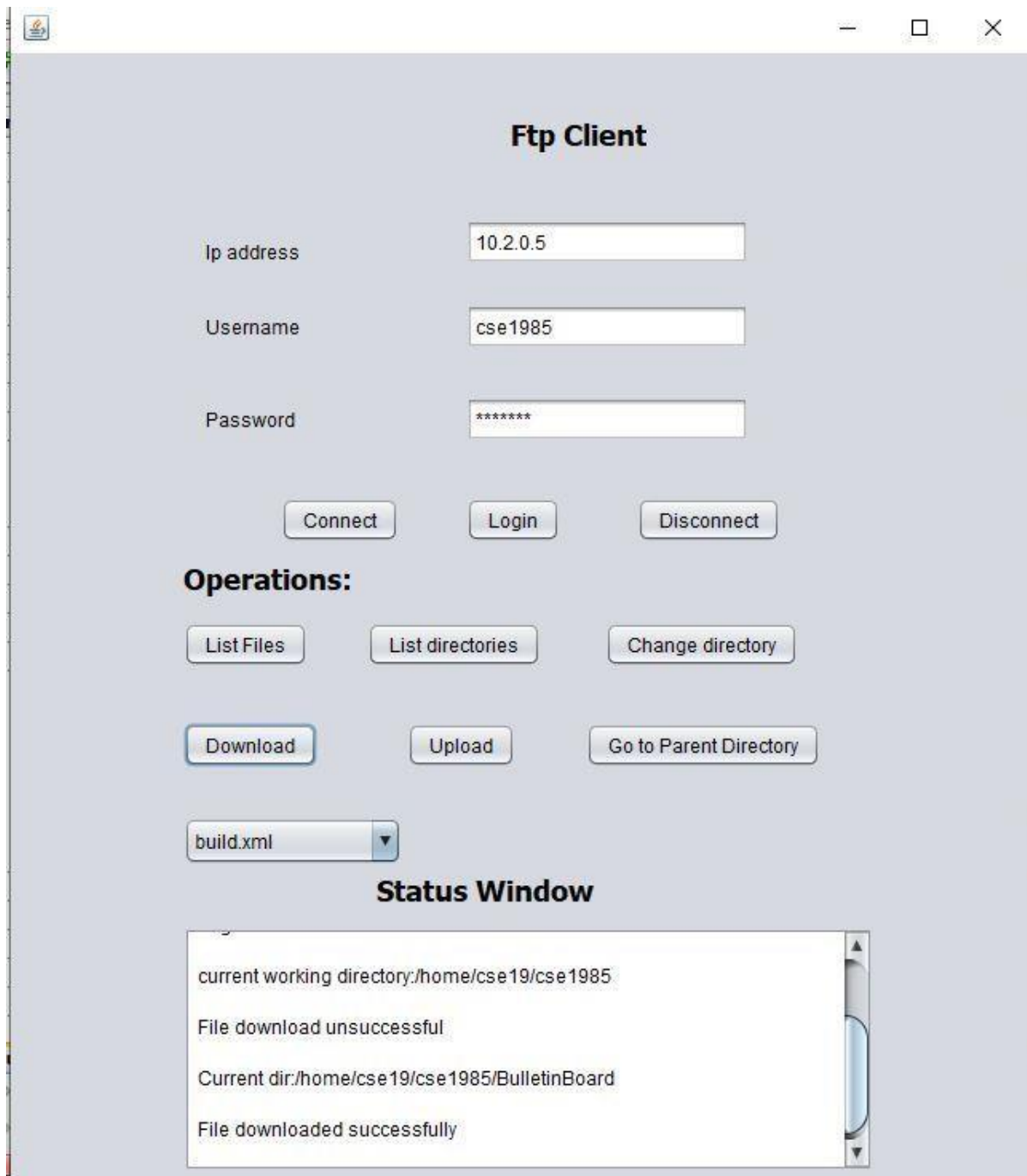
Classes and Methods Used:

- FTPClient
 - FTPClient()
 - connect()
 - getReplyCode()
 - login()
 - listFiles()
 - logout()
 - disconnect()
 - retrieveFile()
- FTPReply
 - isPositiveCompletion()
- FTPFile
 - getName()

Program:

```
package ftpserverclientby;
import java.io.*;
import javax.swing.JFileChooser;
import org.apache.commons.net.ftp.*;
import org.apache.commons.net.ftp.FTPClient;
public class FTPClientInterface extends javax.swing.JFrame {
    FTPClient ftpc;
    private void listFilesButtonActionPerformed(java.awt.event.ActionEvent evt) {
        String ip = IPField.getText();
        String username = usernameField.getText();
        String password = new String(passwordField.getPassword());
        try {
            ftpc = new FTPClient();
            ftpc.connect(ip);
            int status = ftpc.getReplyCode();
            if (FTPReply.isPositiveCompletion(status)) {
                statusWindowArea.append("Connected to " + ip + "\n");
                if (ftpc.login(username, password)) {
                    statusWindowArea.append("User " + username + " logged in to " + ip + "\n");
                    statusWindowArea.append("Current directory is " + ftpc.printWorkingDirectory() + "\n");
                    FTPFile[] files = ftpc.listFiles();
                    dropdownBox.removeAllItems();
                    for (FTPFile file : files) {
                        dropdownBox.addItem(file.getName());
                        statusWindowArea.append(">" + file.getName() + "\n");
                    }
                }
            }
            else {
                statusWindowArea.append("Invalid Credentials\n");
            }
        }
        else {
            statusWindowArea.append("Connection failed\n");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
private void downloadButtonActionPerformed(java.awt.event.ActionEvent evt) {
    try {
        String filename = (String) dropdownBox.getSelectedItem();
        File fname = new File(filename);
        OutputStream os = new BufferedOutputStream(new FileOutputStream(fname));
        if (ftpc.retrieveFile(filename, os)) {
            statusWindowArea.append("Successfully downloaded\n");
        }
        else {
            statusWindowArea.append("Couldn't download\n");
        }
        os.close();
        ftpc.logout();
        statusWindowArea.append("User logged out\n");
        ftpc.disconnect();
        statusWindowArea.append("Disconnected\n");
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}
```

Output Images:



The image shows a screenshot of a web-based FTP client application. The window has a title bar with standard minimize, maximize, and close buttons. The main content area is titled "Ftp Client" and contains several input fields and buttons. The "Ip address" field is set to "10.2.0.5", the "Username" field is set to "cse1985", and the "Password" field is masked with "*****". Below these fields are three buttons: "Connect", "Login", and "Disconnect". A section titled "Operations:" contains six buttons: "List Files", "List directories", "Change directory", "Download", "Upload", and "Go to Parent Directory". Below the operations section is a dropdown menu showing "build.xml". At the bottom of the window is a "Status Window" which displays the following text: "current working directory:/home/cse19/cse1985", "File download unsuccessful", "Current dir:/home/cse19/cse1985/BulletinBoard", and "File downloaded successfully".

Ftp Client

Ip address: 10.2.0.5

Username: cse1985

Password: *****

Connect Login Disconnect

Operations:

List Files List directories Change directory

Download Upload Go to Parent Directory

build.xml

Status Window

current working directory:/home/cse19/cse1985

File download unsuccessful

Current dir:/home/cse19/cse1985/BulletinBoard

File downloaded successfully

PROGRAM - 13

Program Statement:

Implement a GUI based FTP Client program to upload a file

Description of the program:

FTP File Upload Operation connects to an FTP Server and uploads the specified file from the specified path on the local system to the remote system running the FTP Server.

This code is for a Java Swing Application that implements a basic FTP client interface. The interface includes a text field for entering the IP address of the FTP server, a text field for entering the username, a password field for entering the password, a button to list files on the server, a dropdown box to select a file to download or upload, a button to upload a file, and a button to download a file.

The listFilesButtonActionPerformed method is called when the "list files" button is pressed. It connects to the FTP server using the IP address, username, and password provided by the user. If the connection is successful and the credentials are valid, it lists the files on the server and displays them in the dropdown box.

The uploadButtonActionPerformed method is called when the "upload" button is pressed. It connects to the FTP server using the IP address, username, and password provided by the user. If the connection is successful and the credentials are valid, it opens a file chooser dialog for the user to select a file to upload. The selected file is then uploaded to the server.

Input:

IP Address – 10.2.0.5

Username – cse22xx

Password – xxxxxxxx

Output:

The files in the ftp server login are listed in the output window text area.

Packages Used:

- java.io.*
- org.apache.commons.net.ftp.*

Classes and Methods Used:

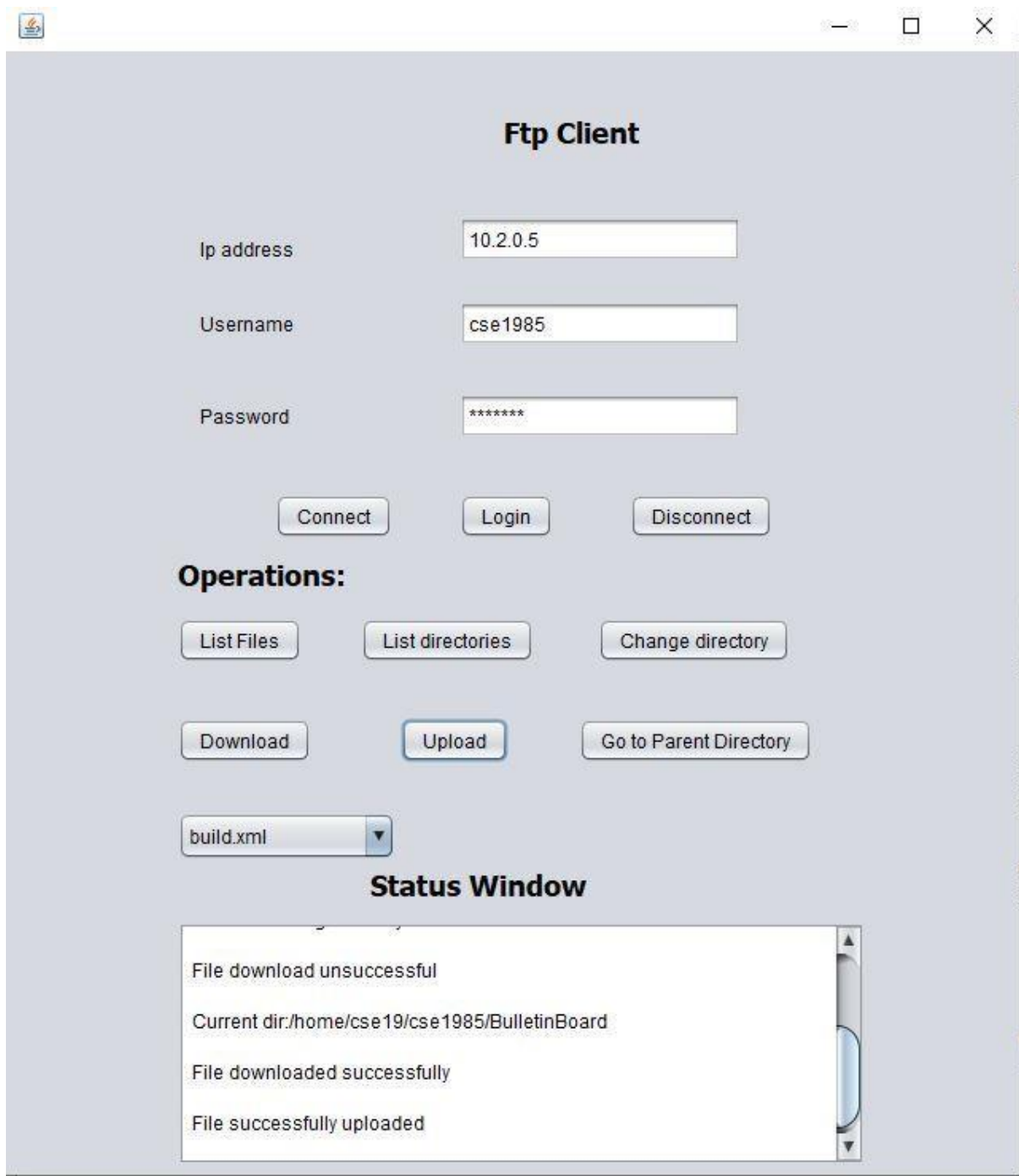
- FTPClient
 - FTPClient()
 - connect()
 - getReplyCode()
 - login()
 - listFiles()
 - logout()
 - disconnect()
 - storeFile()
- FTPReply
 - isPositiveCompletion()
- FTPFile
 - getName()

Program:

```
package ftpserverclientby;
import java.io.*;
import javax.swing.JFileChooser;
import org.apache.commons.net.ftp.*;
import org.apache.commons.net.ftp.FTPClient;
public class FTPClientInterface extends javax.swing.JFrame {
    FTPClient ftpc;          // required at class level
    JFileChooser fileChooser; // required at class level
    private void listFilesButtonActionPerformed(java.awt.event.ActionEvent evt) {
        String ip = IPField.getText();
        String username = usernameField.getText();
        String password = new String(passwordField.getPassword());
        try {
            ftpc = new FTPClient();
            ftpc.connect(ip);
            int status = ftpc.getReplyCode();
            if (FTPReply.isPositiveCompletion(status)) {
                statusWindowArea.append("Connected to " + ip + "\n");
                if (ftpc.login(username, password)) {
                    statusWindowArea.append("User " + username + " logged in to " + ip + "\n");
                    statusWindowArea.append("Current directory is " + ftpc.printWorkingDirectory() + "\n");
                    FTPFile[] files = ftpc.listFiles();
                    dropdownBox.removeAllItems();
                    for (FTPFile file : files) {
                        dropdownBox.addItem(file.getName());
                        statusWindowArea.append(">" + file.getName() + "\n");
                    }
                } else {
                    statusWindowArea.append("Invalid Credentials\n");
                }
            } else {
                statusWindowArea.append("Connection Failed\n");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    private void uploadButtonActionPerformed(java.awt.event.ActionEvent evt) {
        String ip = IPField.getText();
        String username = usernameField.getText();
        String password = new String(passwordField.getPassword());
        try {
            ftpc = new FTPClient();
            ftpc.connect(ip);
            int status = ftpc.getReplyCode();
            if (FTPReply.isPositiveCompletion(status)) {
                statusWindowArea.append("Connected to " + ip + "\n");
                if (ftpc.login(username, password)) {
                    statusWindowArea.append("User " + username + " logged in to " + ip + "\n");
                    statusWindowArea.append("Current directory is " + ftpc.printWorkingDirectory() + "\n");
                    fileChooser = new JFileChooser();
                    fileChooser.showOpenDialog(null);
                    File file = fileChooser.getSelectedFile();
                    InputStream is = new BufferedInputStream(new FileInputStream(file));
                    if (ftpc.storeFile(file.getName(), is)) {
                        statusWindowArea.append("Successfully uploaded\n");
                    } else {
                        statusWindowArea.append("Couldn't upload\n");
                    }
                }
            }
        }
    }
}
```

```
        }
        is.close();
        ftpc.logout();
        ftpc.disconnect();
    } else {
        statusWindowArea.append("Invalid Credentials\n");
    }
} else {
    statusWindowArea.append("Connection Failed\n");
}
} catch (Exception e) {
    e.printStackTrace();
}
}
}
```

Output Images:



The image shows a screenshot of a Java-based FTP client application window titled "Ftp Client". The window has a standard Mac OS X title bar with a red close button, a yellow maximize button, and a green window control button. The main interface is light gray and contains several input fields and buttons. At the top, the title "Ftp Client" is centered. Below it, there are three input fields: "Ip address" with the value "10.2.0.5", "Username" with the value "cse1985", and "Password" with masked characters "*****". Below these fields are three buttons: "Connect", "Login", and "Disconnect". A section titled "Operations:" follows, containing six buttons: "List Files", "List directories", "Change directory", "Download", "Upload" (which is highlighted with a blue border), and "Go to Parent Directory". Below the buttons is a file list box showing "build.xml" with a dropdown arrow. At the bottom, there is a "Status Window" section with a white background and a vertical scrollbar, containing four status messages: "File download unsuccessful", "Current dir:/home/cse19/cse1985/BulletinBoard", "File downloaded successfully", and "File successfully uploaded".

Ftp Client

Ip address: 10.2.0.5

Username: cse1985

Password: *****

Connect Login Disconnect

Operations:

List Files List directories Change directory

Download Upload Go to Parent Directory

build.xml ▼

Status Window

File download unsuccessful

Current dir:/home/cse19/cse1985/BulletinBoard

File downloaded successfully

File successfully uploaded

PROGRAM – 14

Program Statement:

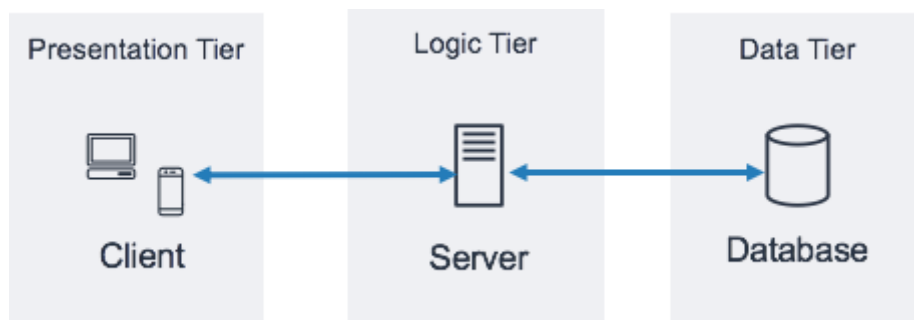
Implement a GUI/CUI based 3-tier application.

Description of the program:

N-tier denotes a software engineering concept used for the design and implementation of software systems using client/server architecture divided into multiple tiers. This decouples design and implementation complexity, thus allowing for the scalability of the deployed system.

A three-tier application is a specific type of n-tier architecture. In the case of three-tier architecture, the tiers are as follows:

- Presentation tier (also known as the user interface or the client application): -
 - The user interaction is managed by the presentation tier, which provides an easy-to-operate front end. It communicates user requests to the application server and presents the results to the user.
- Business logic tier (also known as the application server): -
 - The business rules are managed by the business tier (also known as the middle tier), which controls and operates the entire application framework. The business rules are removed from the client and are executed in the application server. The application server ensures
 - that the business rules are processed correctly. It also serves as an intermediary between the client application and database server.
- Data storage tier (also known as the database server): -
 - The underlying data is stored and served by the data storage tier, also known as data persistence. It serves data whenever the application server sends a request to it.



Our 3-tier application consists of a database server, backend server in this case java, and a frontend User interface, which in this case is Java Swing Interface. The interface includes a text field for entering the username, a password field for entering the password, a text field to enter the fields/columns to be used in the query, a button to list tables in the database, a dropdown box to select a table to query, and a button to display the contents of table using the select query.

The `listTablesBtnActionPerformed` method is called when the "list tables" button is pressed. It connects to the MySQL server and lists the tables in the database on the server and displays them in the dropdown box.

The `displayTableBtnActionPerformed` method is called when the "Display Tables" button is pressed. It connects to the MySQL server and displays the content in the selected table of the dropdown box with the specified fields (default *).

Input:

Username – cse22xx

Password – xxxxxxxx

Fields – * or any column name(s)

Output:

The tables in the database are listed, and their content is displayed.

Packages Used:

- `java.sql.*`

Classes and Methods Used:

- `Connection`
 - `createStatement()`
 - `close()`
- `DriverManager`
 - `getConnection()`
- `Statement`
 - `executeQuery()`
- `ResultSet`
 - `getMetaData()`
 - `next()`
 - `getString()`
- `ResultSetMetaData`
 - `getColumnCount()`
 - `getColumnName()`

Program:

```
import java.sql.*;
public class NewJFrame extends javax.swing.JFrame {
    Connection con;
    private void connectBtnActionPerformed(java.awt.event.ActionEvent evt) {
        String user = jTextField1.getText();
        String password = new String(jPasswordField1.getPassword());
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con = DriverManager.getConnection("jdbc:mysql://10.2.4.6:3306/student", user, password);
            jTextArea1.append("Connection Established\n");
            connectBtn.setEnabled(false);
            disconnectBtn.setEnabled(true);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    private void disconnectBtnActionPerformed(java.awt.event.ActionEvent evt) {
        try {
            con.close();
            jTextArea1.append("Connection closed\n");
            connectBtn.setEnabled(true);
            disconnectBtn.setEnabled(false);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    private void listTablesBtnActionPerformed(java.awt.event.ActionEvent evt) {
        try {
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("show tables");
            jComboBox1.removeAllItems();
            while (rs.next()) {
                jTextArea1.append(rs.getString(1) + "\n");
                jComboBox1.addItem(rs.getString(1));
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
    private void displayTableBtnActionPerformed(java.awt.event.ActionEvent evt) {
        String str = (String) jComboBox1.getSelectedItem();
        String fild = jTextField2.getText();
        try {
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("select " + fild + " from " + str);
            jTextArea1.append("\n");
            ResultSetMetaData rsmd = rs.getMetaData();
            for (int i = 1; i <= rsmd.getColumnCount(); i++) {
                jTextArea1.append(rsmd.getColumnName(i) + "\t");
            }
            jTextArea1.append("\n");
            while (rs.next()) {
                for (int i = 1; i <= rsmd.getColumnCount(); i++) {
                    jTextArea1.append(rs.getString(i) + "\t");
                }
                jTextArea1.append("\n");
            }
        }
    }
}
```

```
    } catch (Exception e) {  
        e.printStackTrace();  
    }  
}  
}
```


Output Images:

The screenshot shows a web application window titled "three tier". It contains a login form with fields for Username, Password, and Fields. The Username field is filled with "cse19306". The Password field is empty. The Fields field is filled with "studdetails". There are buttons for "Connect", "Disconnect", "List Tables:", and "Display Tables:". The "List Tables:" button is highlighted. Below the form, a text area displays the following information:

connection Established
studdetails

rollno	name1	branch	address
21	adams	cse	mjcet
22	turner	cse	mjcet
25	march	ece	mjcet
26	april	ece	mjcet
41	mongo	aids	mjcet

Connection closed

PROGRAM – 15

Program Statement:

NFS Case Study – mounting of specified directory from Linux server to Linux client, windows client.

Description of the program:

NFS, stands for Network File System, is a server-client protocol used for sharing files between systems. NFS enables you to mount a remote share locally. You can then directly access any of the files on that remote share.

The basic idea behind NFS is that each file server provides a standardized view of its local file system. This approach has been adopted for other distributed files systems as well. NFS allows a heterogeneous collection of processes to run on different operating systems and machines, and share a common file system.

The model underlying NFS and similar systems is that of a remote file service. In this model, clients are offered transparent access to a file system that is managed by a remote server. However, clients are normally unaware of the actual location of files. Instead, they are offered an interface to a file system that is similar to the interface offered by a conventional local file system. In particular, the client is offered only an interface containing various file operations, but the server is responsible for implementing those operations. This model is therefore also referred to as the remote access model.

A client accesses the file system using the system calls provided by its local operating system. However, the local UNIX file system interface is replaced by an interface to the Virtual File System (VFS), which by now is a de facto standard for interfacing to different (distributed) file systems.

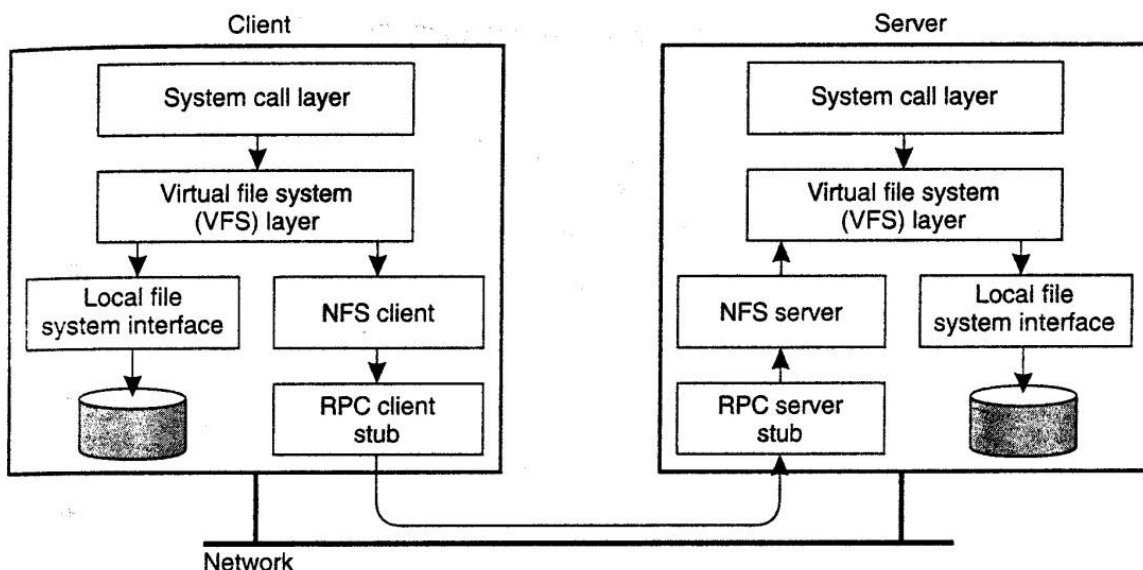
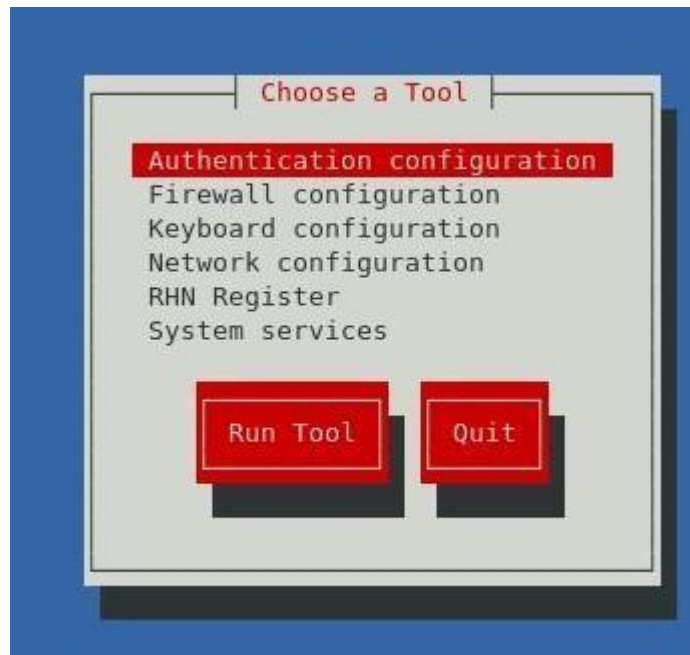


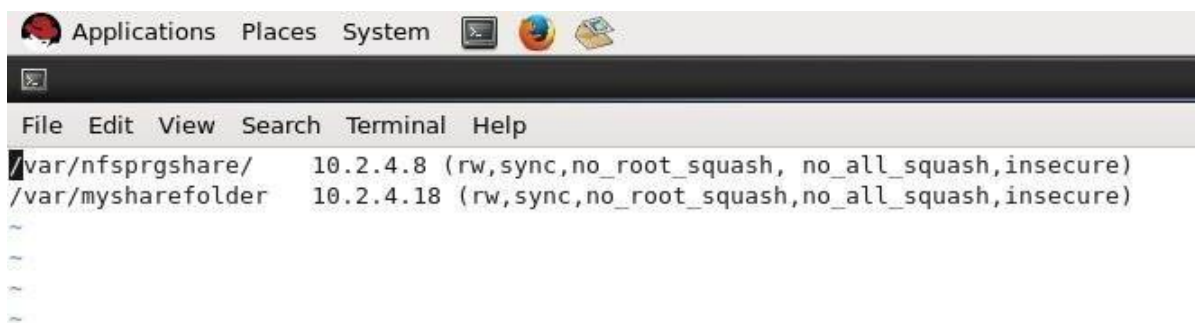
Figure 11-2. The basic NFS architecture for UNIX systems.

Procedure:

Linux Server:



- Install linux full-fledged – RHEL (Red hat enterprise linux)
- Open terminal, type
`>setup`
- In setup select system services
- Ensure NFS service is enabled
- Create a directory in root directory under var sub-directory
`>mkdir /var/mySharedFolder`
- Give permissions to the folder
`>chmod 777 mySharedFolder`
- Open exports file
`>vi /etc/exports`
- Mention the shared folder path , client Ip that can mount it and the different permissions by adding a line at the end file as below
`>/var/mySharedFolder 10.2.0.14 ( rw, sync, no_root_squash, no_all_squash, insecure)`
- Save and exit the file
- Then restart the NFS service to apply the changes with the command
`>service nfs restart`



```
Applications Places System root@localhost:/
File Edit View Search Terminal Help
[root@localhost ~]# setup
[root@localhost ~]# ls
'          del_exam.sh  Desktop  install.log  Pictures  Videos
anaconda-ks.cfg  del_user.sh  Documents install.log.syslog Public
create_user.sh   del_user.sh  Downloads Music      Templates
[root@localhost ~]# cd ..
[root@localhost /]# ls
bin  cgroup  etc  lib  lost+found  misc  net  proc  sbin  srv  tmp  usr
boot  dev  home  lib64  media  mnt  opt  root  selinux  sys  usr
[root@localhost /]# mkdir /var/mysharefolder
[root@localhost /]# chmod 777 /var/mysharefolder
[root@localhost /]# vi /etc/exports
[root@localhost /]# service nfs restart
Shutting down NFS daemon: [ OK ]
Shutting down NFS mountd: [ OK ]
Shutting down NFS quotas: [ OK ]
Shutting down NFS services: [ OK ]
Shutting down RPC idmapd: [ OK ]
Starting NFS services: exportfs: No options for /var/nfsprgshare/ 10.2.4.8: suggest 10.2.4.8(sync) to avoid warning
exportfs: No host name given with /var/nfsprgshare/ (rw,sync,no_root_squash,, suggest *(rw,sync,no_root_squash, to avoid warning
exportfs: /etc/exports:1: syntax error: bad option list [FAILED]
Starting NFS quotas: [ OK ]
Starting NFS mountd: [ OK ]
Starting NFS daemon: [ OK ]
Starting RPC idmapd: [ OK ]
[root@localhost /]#
```

Linux-client: -

- Similar to server ensure that the NFS service is enabled.
- Make a directory
>mkdir myMountFolder
- Mount the shared folder of the server on the client machine

Syntax: -

>mount -t nfs server-IP:folder-path-on-server folder-path-on-client

Ex: -

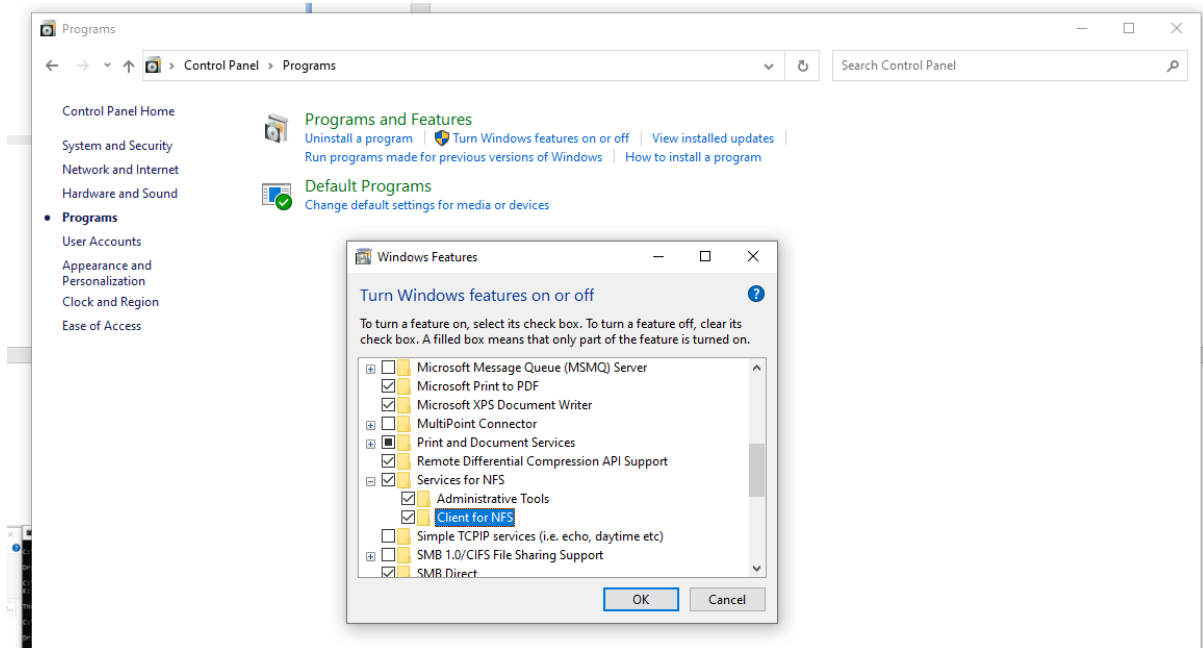
>mount -t nfs 10.2.0.5:/var/mySharedFolder /myMountFolder

- Now you can access the mounted folder by changing directory to myMountFolder

```
Applications Places System root@localhost:~/Desktop/my
File Edit View Search Terminal Help
[root@localhost Desktop]# ls
gnome-terminal.desktop  Packages
[root@localhost Desktop]# mkdir mymount
[root@localhost Desktop]# mount -t nfs 10.2.0.5:/var/cshare2022 mymount
[root@localhost Desktop]# ls
gnome-terminal.desktop  Packages
[root@localhost Desktop]# cd mymount
[root@localhost mymount]# ls
ChatServer1  commons-net-3.6.jar  NewJFrame.java  nfs.txt  System Volume Information  tcpechocs  weka.ppt
[root@localhost mymount]#
```

Windows client:-

- Go to control panel>programs>turn windows features on or off
- Search for “services for NFS”
- Check “client for NFS” and “administrative tools” in it.
- Click ok, after changes apply, restart system



Windows client-CMD:

- Run CMD as Admin
- Put the below command to mount

Syntax:-

>mount -o anon [\\server-ip\folder-path\](#) drive-letter

Ex:-

>mount -o anon [\\10.2.0.5\var\mySharedFolder\](#) K:

- Then access the drive in CMD by typing,

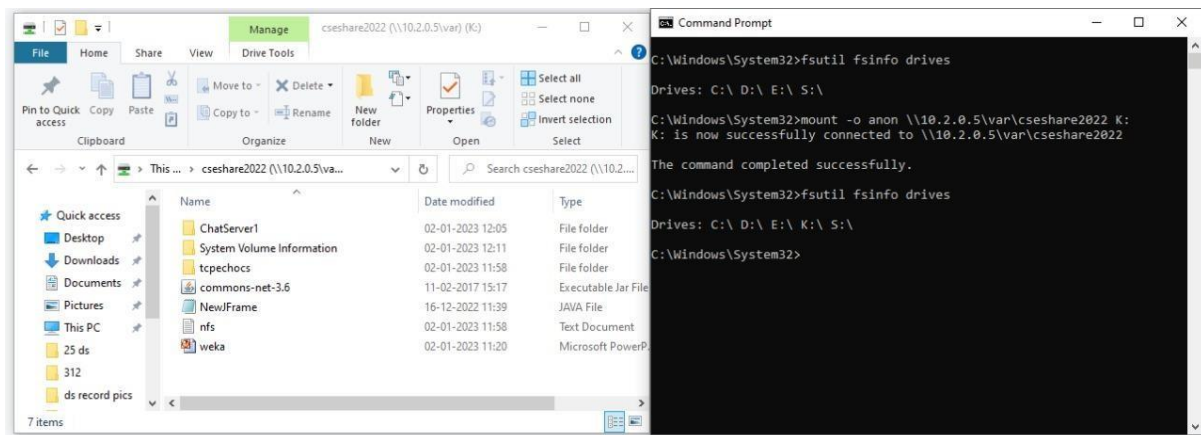
>K:

K:>dir

.....

.....

.....folder contents are displayed



Windows client-GUI:

- Click on This PC
- Above address bar, in the computer tab select “Map network drive”
- Select a drive letter which has not been already used.
- Specify folder on server which is to be shared as
[\\10.2.0.5\var\mySharedFolder](#)
- Check remount at sign-in
- Click finish

